



Programação II

Programação de Computadores II

2025/2026 - João Rebelo - ISTEC - P.O.O. e Modelação



Revisão de Algoritmia

Codificar em Pseudo-código;

Elaboração de Fluxogramas;

Conversão para Linguagens de Programação;

Algoritmia

Um ALGORITMO deve sempre considerar:

Uma ou Mais Entradas de Dados - As matérias-primas → Quantidades inicialmente especificadas (por exemplo, através de instruções de leitura).

Uma ou Mais Saídas - O produto final → Uma ou mais saídas (habitualmente por instruções de escrita em qualquer suporte, papel, monitor, ficheiro, etc.)

Número Finito de Passos → A execução deve prever sempre num número finito de passos. Não é concebível obter um produto final a partir de um processo que não termina

Precisão → Todos os passos do algoritmo devem ter um significado preciso, sem ambiguidades, indicando exatamente o que deve ser feito, bem como os resultados obtidos perante as mesmas condições de entrada.

Eficácia → O processo de um algoritmo, deve conduzir à resolução do problema proposto

Eficiência → Se um algoritmo **A** para resolver um problema **X** proposto, demorar menos tempo a executar ou consumir menos recursos que outro algoritmo **B**, então o algoritmo **A** é considerado mais eficiente que o **B** (no aspeto avaliado) embora sejam ambos eficazes (ou seja, ambos resolvem o problema).

Algoritmia - Operações

Leitura de Dados → Ler()

Escrita de Informação → Escrever (Informação)

Atribuição → Colocar numa variável um determinado valor (o que estava, desaparece)

Operação Aritmética → Incrementar ou Decrementar o valor de uma variável

Instrução Condicional → Executar se Determinada Condição for verdadeira

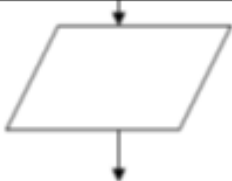
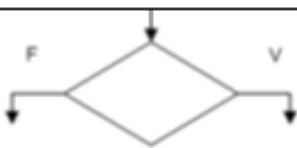
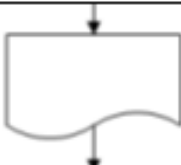
Teste de Validade de Condições → Através de tabelas de verdade

Operações Cíclicas Pré-determinadas → Ciclos PARA

Operações Cíclicas Condicionais → Fazer ENQUANTO ou Fazer ATÉ

Algoritmia - Fluxograma

Descrição Gráfica de um algoritmo. Representa as operações de entrada, saída, decisão e processamento de um algoritmo

Símbolo	Significado / Utilização
	Representa sempre o ponto único de entrada e ponto único de saída de um algoritmo
	Representa a Leitura de Dados ou Escrita de Informação
	Representa uma atribuição ou processamento
	Representa uma estrutura de decisão ou condicional
	Alternativa para escrita de informação

Algoritmia - Exemplo de Aula

Algoritmo que descreve o processo de cálculo de uma tabuada N

Algoritmo: Tabuada

Descrição: Este algoritmo calcula a tabuada de um número solicitado ao utilizador

[Variáveis Utilizadas]

contador <- 1

tabuada <- 0

resultado <- 0

[Introdução dos Dados]

Escrever("Digita o número do qual queres ver a tabuada, p.e. 9:")

tabuada <- Ler()

[Calculo da Tabuada e Escrita dos Resultados]

Enquanto contador <= 10

 resultado <- contador * tabuada

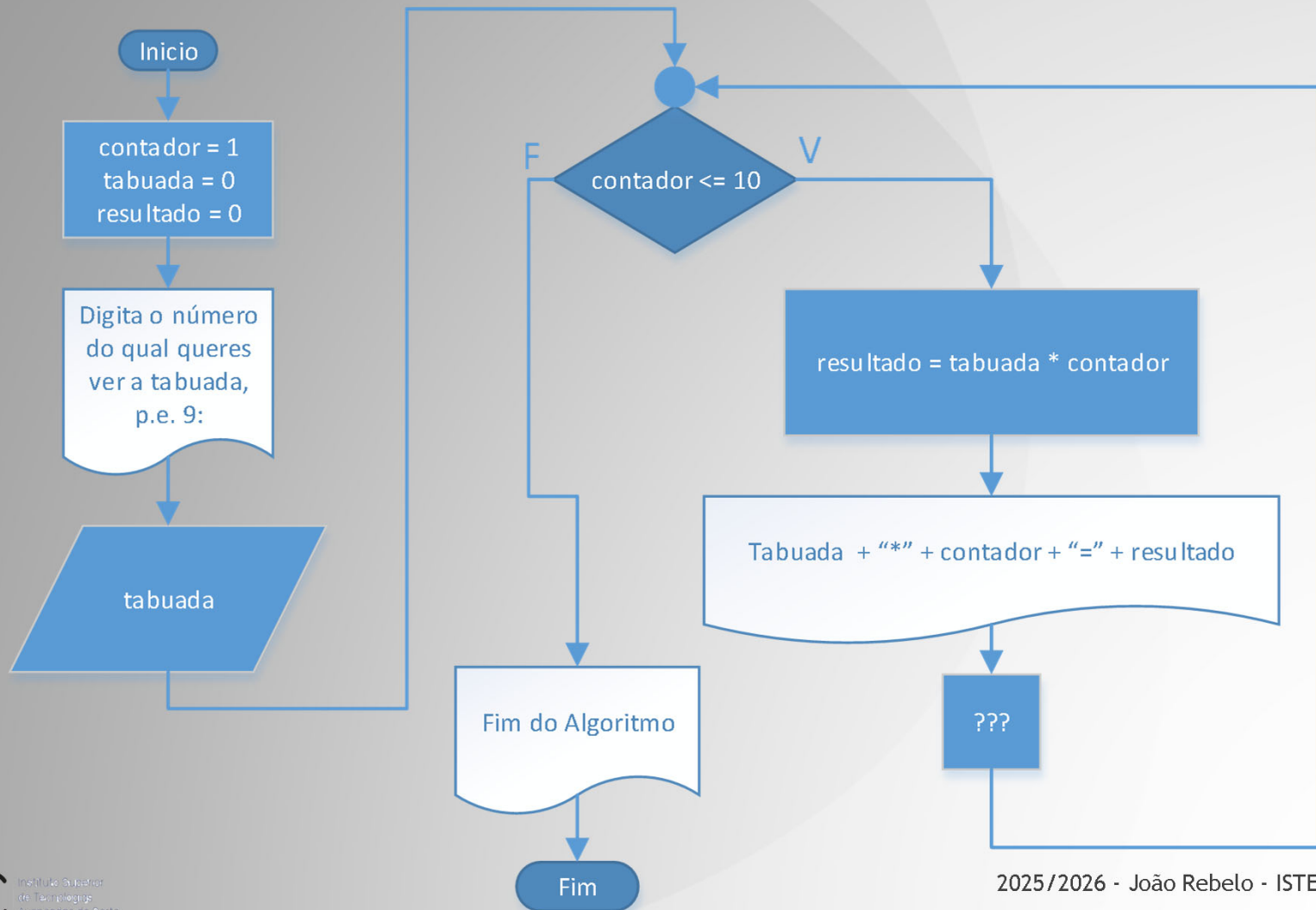
 Escrever (tabuada + "*" + contador + " = " + resultado)

 [este ciclo tem um "bug"... qual é?]

Escrever("Fim do Algoritmo")

Algoritmia - Exemplo de Aula

Fluxograma do Algoritmo elaborado



Algoritmia - Exemplo de Aula

A conversão para C, C++, C#, Java, Javascript... etc.

```
int contador = 1;
int tabuada = 0;
int resultado = 0;

Console.WriteLine("Digita o número do qual queres ver a tabuada, p.e. 9:");
tabuada = Convert.ToInt32(Console.ReadLine());

while (contador <= 10) {
    resultado = tabuada * contador;
    Console.WriteLine(tabuada + "*" + contador + " = " + resultado);
    contador++;
}

Console.WriteLine("Fim do Algoritmo");
Console.ReadLine();
```


Algoritmia - Exemplo de Aula

... e em Python:

```
tabuada.py - C:\Users\jr.CESAE-DOM\Desktop\tabuada.py (3.13.0)
File Edit Format Run Options Window Help

contador = 1
tabuada = 0
resultado = 0

print("Digita o número do qual queres ver a tabuada, p.e. 9:")
tabuada = int(input())

while contador <= 10:
    resultado = tabuada * contador
    print(str(tabuada) + "*" + str(contador) + " = " + str(resultado))
    contador += 1

print("Fim do Algoritmo")
input()
```

Algoritmia - Exemplo de Aula

... e já agora em R:

```
contador <- 1
tabuada <- 0
resultado <- 0

cat("Digita o número do qual queres ver a tabuada, p.e. 9:\n")
tabuada <- as.integer(readline())

while (contador <= 10) {
  resultado <- tabuada * contador
  cat(tabuada, "*", contador, "=", resultado, "\n")
  contador <- contador + 1
}

cat("Fim do Algoritmo\n")
readline()
```

Algoritmia - Q.A.

Elabore um algoritmo em pseudo-código, que leia 4 números distintos, e da forma mais eficiente possível, mostre o menor e o maior. **Sem vetores e ciclos; apenas 4 variáveis (além de outras que considere necessárias)**

Elabore ainda o respetivo fluxograma e a versão C# (preferencialmente) ou Python.



Programação Orientada a Objetos

Programação, Análise e Modelação Orientadas a Objetos

O Problema “Object Oriented”

- Ver um problema como um conjunto de objetos relacionados em vez de um conjunto de instruções a serem executadas em cima de estruturas de dados.
- **Os objetos (instanciados a partir de classes):**
 - Têm características relevantes para o problema → Atributos
 - Sabem efetuar ações relevantes para o problema → Métodos
- **As regras:**
 - Nenhum atributo deve ser exposto publicamente. Considerar apenas a exposição a classes derivadas quando aplicável
 - Todos os atributos devem ser controlados por métodos.
 - A classe deve ser agnóstica ao ambiente onde é utilizada

Classes Agnósticas???

Em T.I. contexto **agnóstico** refere-se a algo (software ou hardware) de utilização geral/genérico/transversal entre vários sistemas.

- Classe Agnóstica - construída de tal forma que pode ser reutilizada em qualquer sistema ou arquitetura, sem qualquer modificação.

COMO ?

- Manter as regras indicadas (atributos privados e apenas modificáveis via métodos).
- Classe nunca lê diretamente dos dispositivos de entrada (ex: teclado) ou escreve diretamente nos dispositivos de saída (ex: monitor)
- A Classe: Aceita dados via métodos e devolve informação via métodos

Um exemplo Evolutivo P1:

```
class ErradaV1 {  
    public int valorPositivo;  
}
```

```
C:\WINDOWS\system...  
Valor Positivo:-30  
Press any key to continue . . .
```

```
s Program {  
    static void Main(string[] args) {  
        ErradaV1 ev1 = new ErradaV1();  
        ev1.valorPositivo = -30;  
        Console.WriteLine("Valor Positivo:" + ev1.valorPositivo);  
    }  
}
```

Publico → Manipulável → Erro Grave: aceita qualquer valor

```
class ErradaV1 {  
    private int valorPositivo;  
}
```

```
s Program {  
    static void Main(string[] args) {  
        ErradaV1 ev1 = new ErradaV1();  
        ev1.valorPositivo = -30;  
        Console.WriteLine("Valor Positivo:" + ev1.valorPositivo);  
    }  
}
```

struct System.Int32
Represents a 32-bit signed integer.
'ErradaV1.valorPositivo' is inaccessible due to its protection level

Privado → Não Manipulável →

Um exemplo Evolutivo P2:

Acrescentemos dois métodos: um para ler do teclado e outro para escrever no monitor

```
class MenosErradaV2 {  
    private int valorPositivo;  
  
    public void lerValorPositivo() {  
        Console.WriteLine("Valor Positivo:");  
        try { valorPositivo = int.Parse(Console.ReadLine()); }  
        catch { valorPositivo = 0; }  
        if (valorPositivo <= 0) {  
            valorPositivo = 0;  
            Console.WriteLine("Deve inserir valores positivos");  
        }  
    }  
  
    public void escreverValorPositivo() {  
        Console.WriteLine("Valor:" + valorPositivo);  
    }  
}
```

```
class Program {  
    static void Main(string[] args) {  
        MenosErradaV2 mev2 = new MenosErradaV2();  
        //ev1.valorPositivo = -30; Impossível  
        mev2.lerValorPositivo();  
        mev2.escreverValorPositivo();  
    }  
}
```

```
cmd Select C:\WINDOWS\system32\cmd.exe  
Valor Positivo:  
-10  
Deve inserir valores positivos  
Valor: 0  
Press any key to continue . . .
```

Atributo Privado → OK

Método para ler e validar um valor originário do teclado → OK

Método para escrever no ecrã o valor guardado no atributo → OK

Um exemplo Evolutivo P3:

... Acabamos de criar uma classe que só pode ser utilizada em programas do tipo “**Console**”

Porque:

- A Classe está a ler diretamente da Console.IN (Readline)
- A Classe está a escrever diretamente na Console.OUT (Writeline)

Não dá para a utilizar noutras arquiteturas, ex: Mobile, WEB, WPF, etc.

Um exemplo Evolutivo P4:

Criamos métodos para “levarem” valores para os atributos

E

Criamos métodos para “trazerem” os valores dos atributos

Na Prática:

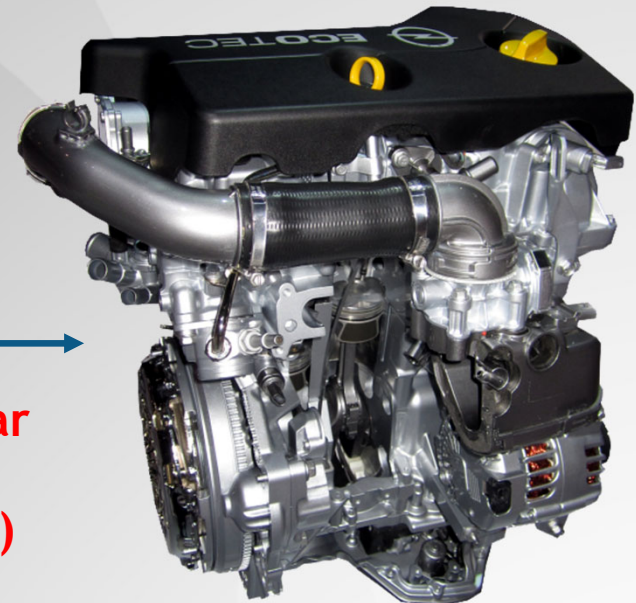
Temos métodos que atribuem valores (SET) aos atributos,
portanto *SETTERS*

Temos métodos que leem os valores (GET) dos atributos,
portanto *GETTERS*

Um exemplo Evolutivo P4: Analogia



Método **Acelerar**
(SET Rotações)



Altera-se o valor do atributo

“Potência”



Método **Ler**
(GET Rotações)

Um exemplo Evolutivo P5:

```
class BoaV1 {  
    private int valorPositivo;  
  
    public void setValorPositivo(string valor) {  
        try { valorPositivo = int.Parse(valor); }  
        catch { valorPositivo = 0; }  
        if (valorPositivo <= 0) {  
            valorPositivo = 0;  
        }  
    }  
  
    public int getValorPositivo() {  
        return valorPositivo;  
    }  
}
```

```
namespace Evol {  
    class Program {  
        static void Main(string[] args) {  
            BoaV1 boa = new BoaV1();  
            //ev1.valorPositivo = -30; Impossível  
            boa.setValorPositivo("-30");  
            int res = boa.getValorPositivo();  
            Console.WriteLine("Valor:" + res);  
        }  
    }  
}
```

```
C:\WINDOWS\system...  
Valor:0  
Press any key to continue . . .
```

Setter

Getter

A Visibilidade dos Membros

- No conceito de Classe, um membro pode ser qualquer característica ou funcionalidade da mesma.
- Qualquer membro, pode estar ou não exposto ao exterior: conceito de visibilidade
- Visibilidade de um membro:
 - **PRIVATE** → apenas acessível no contexto da própria classe
 - **PROTECTED** → acessível à própria classe e derivadas (?)
 - **PUBLIC** → acessível externamente
- REFORÇO da REGRAS PRINCIPAIS:
 - Atributos sempre privados ou protegidos (quando aplicável)
 - Métodos controlam o valor do atributo
 - As classes devem ser independentes da plataforma
- PORTANTO:

Método → forma controlado de expor um atributo ao exterior

Demonstração de Visibilidade

- Criar duas classes que permitam guardar:
 - Um nome com tamanho máximo de 30 caracteres
 - Um ano de nascimento entre 1900 e o ano atual
 - Tenha - para já - um único método para visualizar a informação:
- ClasseErrada:
 - Criar os atributos como públicos
 - Inicializá-los (acesso externo)
 - Mostrá-los
 - Qual o resultado? Conseguimos impor regras?
- ClasseCerta:
 - Criar os atributos como privados
 - Criar um “setter” para o nome e um “setter” para o ano, que imponham as regras pretendidas
 - Inicializar com os mesmos valores anteriores
- E agora, qual o resultado?

Organizar a confusão

- Atributos controlados por métodos
- Convencionar nomes para os atributos
- Convencionar nomes para os métodos que os manipulam
- Tendo em conta que:
 - Há métodos que “levam” matéria prima para os atributos dos objetos: os “**modificadores**” (o acelerador do carro)
 - Há métodos que “leem” o valor colocado nos atributos dos objetos, sem os afetar: os “**leitores**” (o conta-rotações do carro)
- Convenção à prova de bala:
 - Modificadores de atributos → setters
 - Leitores de atributos → getters
- Ter ainda em conta que podemos ter outro tipo de métodos que fazem algo no objeto sem ser apenas manipular/ler atributos, tal como um cálculo p.e.

O mau e o bom 😊

```
class Pessoa {  
    private string Nome;  
  
    public void Manipular(string nome) {  
        Nome = nome;  
    }  
    public string Ler() {  
        return Nome;  
    }  
}
```

O MAU:

- Como convencionar os métodos para múltiplos atributos?
- Facilmente se confunde e troca o **nome** por **Nome** e vice-versa

```
class Pessoa {  
    private string _nome;  
  
    public void setNome(string nome) {  
        _nome = nome;  
    }  
    public string getNome() {  
        return _nome;  
    }  
}
```

O BOM:

- Set e Get como prefixo do nome do atributo
- Impossível confundir o atributo **_nome** e o parâmetro **nome**
- Único Senão: os adeptos do **this** ficam zangados

Último Ponto: Instanciar e Inicializar

- Instanciar em OOP → criar um objeto a partir da classe
- Inicializar em OOP → o objeto é instanciado e devidamente inicializado com valores-padrão que respeitem as regras de negócio.
- Exemplo - A empresa X quer criar um calculador de IMC, mas determina que:
 - O valor da altura → $0,10 \leq \text{ALTURA (metros)} \leq 3$
 - O valor do → $0,10 \leq \text{PESO (quilos)} \leq 500$
 - Vamos criar a class IMC que implemente estas regras e implemente o método de cálculo do imc
 - Questão:
 - Qual o resultado deste código:

```
IMC imcCalc = new IMC();  
decimal imc = imcCalc.Calcular();  
Console.WriteLine ("imc = " + imc);
```

Último Ponto: Instanciar e Inicializar

- Como a formula do imc = peso / altura²
 - Se utilizarmos os valores “default” para atributos numéricos (=0)
 - E fizermos logo o calculo sem passar utilizar os setters
 - Temos: ERRO → Impossível dividir por zero
- SOLUÇÃO:
 - Garantir que o objeto é devidamente inicializado após a instanciação
 - Mesmo que façamos o cálculo logo após a instanciação, não vamos ter erro pois o objeto tem valores “estáveis” e válidos
- COMO:
 - Configurar **CONSTRUCTORS**

Último Ponto: o Constructor

- É um método especial que é invocado na instanciação de um objeto
- No caso da maior parte das L.O.O. (C++, C#, Java, etc.)
 - É um método público
 - Não tem qualquer tipo (sim, nem void)
 - Tem o mesmo nome da classe
 - Deve ter o cuidado de inicializar todos os atributos da classe
 - De forma direta se não existirem setters
 - Utilizando os setters caso estejam definidos

Constructor



```
class Pessoa {  
    private string _nome;  
    public void setNome(string nome) {  
        _nome = nome;  
    }  
    public string getNome() { return _nome; }  
    public Pessoa() {  
        setNome("");  
    }  
}
```

Relações entre objetos:

Existem muitos tipos de relações entre classes de objetos, mas as mais comuns são:

- Herança → classe derivada (subclasse) herda características e métodos da classe base (superclasse).
 - Por exemplo, podemos dizer que um “médico”, um “enfermeiro” ou um “paciente” de uma clínica, são especializações de classe genérica “pessoa”
 - Diz-se que o “médico”, “enfermeiro” e “paciente” são subclasses da classe base “pessoa”
- Agregação → um objeto da classe X agrega *temporariamente* múltiplos objetos de outro tipo de classe Y.
 - Relação de 1 para vários (1 agregador / múltiplos agregados)
 - “Eliminando” o agregador não se eliminam os agregados
 - “Eliminando” um ou mais agregados não se elimina o agregador

Exemplos:

- 1 Hotel com múltiplos Clientes
- 1 Carro de Compras com múltiplos Produtos

Relações entre objetos (cont.):

- Composição → um objeto da classe X é composto por múltiplos objetos de outro tipo de classe Y.
 - Relação de 1 para vários (1 principal / múltiplos ligados)
 - Sempre que ocorre uma (ou as duas) destas circunstâncias:
 - “Eliminando” o principal eliminam-se os componentes
 - “Eliminando” um ou mais dos componentes invalida o principal
 - Exemplos:
 - 1 Carro com múltiplas rodas
 - 1 Fatura com múltiplos produtos faturados
- Associação → um objeto da classe X está associado a outro objeto da classe Y
 - Normalmente uma relação de 1 para 1
 - Normalmente uma relação independente (eliminando 1 deles, o outro continua a fazer sentido)
 - O sistema só é útil e funcional quando X está associado a Y
 - Exemplos:
 - 1 funcionário está ligado a uma empresa
 - 1 produto está ligado a um fornecedor
 - 1 condutor está ligado a um veículo

Relações entre objetos (cont.):

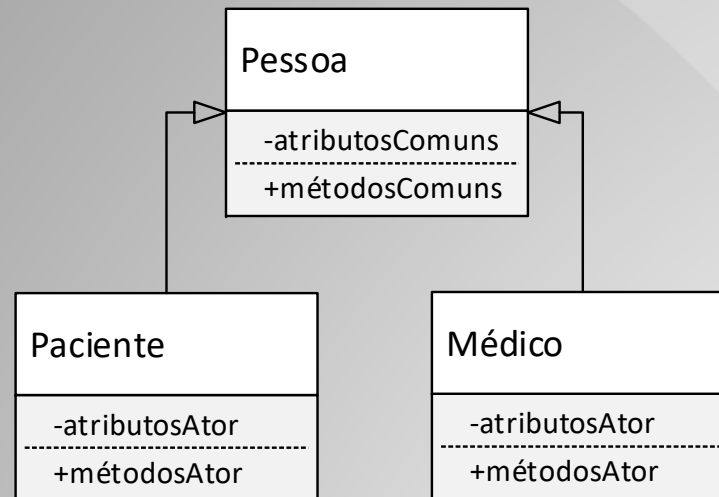
- Composição vs. Agregação vs. Associação → a grande confusão (para rimar 😊)
- Questionar da seguinte forma se dúvida é entre composição e agregação:
 - O TODO pode ser eliminado sem invalidar os elementos?
 - Os elementos podem ser utilizados em mais que um TODO?
 - Destruindo qualquer um dos elementos não implica a inutilização do TODO?
 - SE SIM A ESTAS 3 QUESTÕES → AGREGAÇÃO SEM QUALQUER DÚVIDA
- Questionar da seguinte forma se dúvida é entre agregação e associação:
 - Um dos objetos é necessário para que o outro seja utilizável/faça sentido?
 - O objeto relacionado faz sentido como um elemento isolado?
 - SE SIM A ESTAS 2 QUESTÕES → ASSOCIAÇÃO SEM QUALQUER DÚVIDA

Relações entre objetos - Modelo

- Da mesma forma que um arquiteto cria um modelo da casa para mostrar ao cliente, o analista deve criar um modelo do sistema para mostrar/validar com o cliente
- Arquitetura → utilizam-se plantas e modelos 3D
- Sistemas → utilizam-se diagramas UML
- UML → “*Unified Modeling Language*”
 - Várias vistas sobre o mesmo sistema (analogia com o google maps que permite ter várias perspetivas do terreno, como por exemplo, a vista de satélite, mapas, *street view*, etc.
 - Composto por muitos tipos de diagramas, mas os mais interessantes para já são:
 - Diagrama de casos de uso (ou “Use Case”) → Funcionalidades
 - Diagrama de classes → Classes e respetivas relações

Relações entre objetos - Modelo

- Representação das Relações em UML
- Herança



- Composição

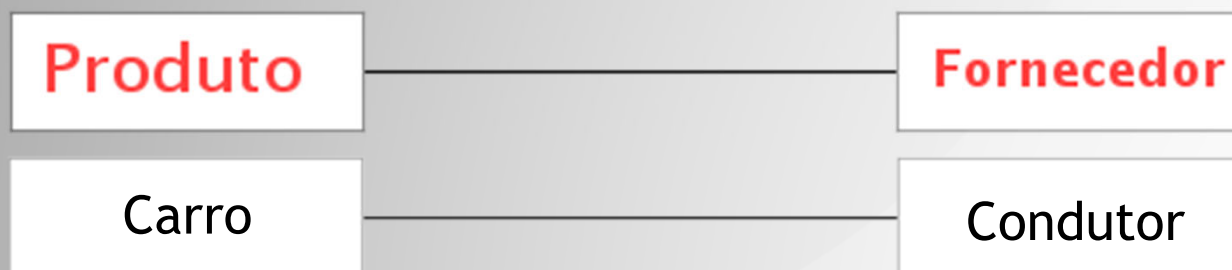


Relações entre objetos - Modelo

- Representação das Relações em UML
- Agregação



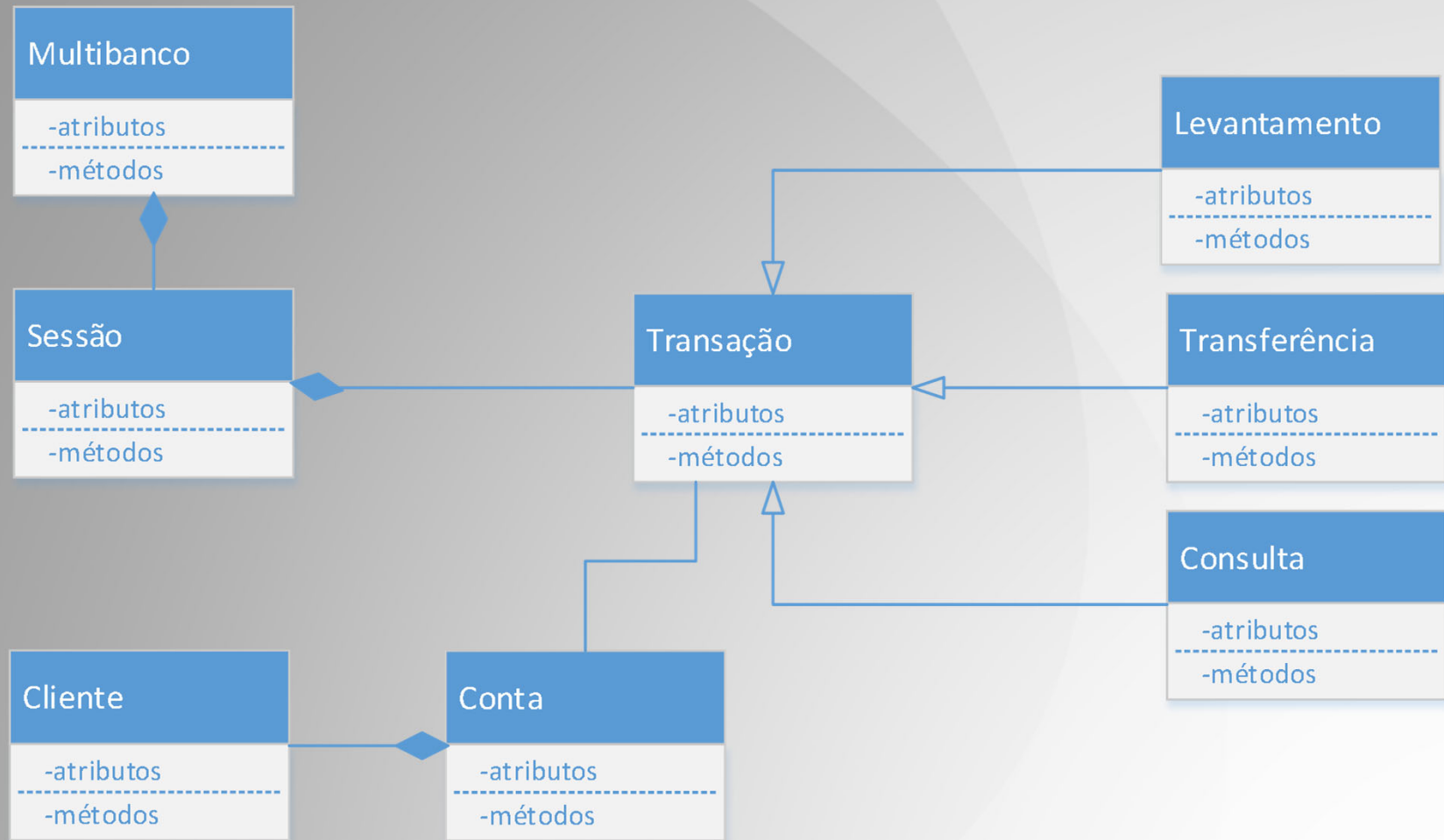
- Associação



Um problema O.O.: Sistema MB

- “Um *Sistema Multibanco*, permite a um *utilizador cliente* de um *Banco* com uma *conta ativa*, efetuar diversas operações, nomeadamente, *Consultar Saldo*, *Efetuar Pagamentos* e *Levantar Dinheiro*, entre outras, no contexto de uma *Sessão*”
- O parágrafo acima (abordagem comum a todos os problemas orientados a objetos), permite identificar desde logo:
 - Possíveis entidades para o problema (*a vermelho*)
 - Possíveis funcionalidades do sistema (*a verde*)
- Existe sempre uma relação entre os objetos de um problema, por exemplo, Herança... vejamos outras

Um possível Modelo de Classes - MB



Para praticar: Escrever um parágrafo descritivo do modelo de classes acima.

Complementar Diagrama Classes

- Enumerar os Atributos (tipo e quantidade)
- Enumerar as Funcionalidades (com a assinatura respetiva)
- Ex: Atributos de uma “Sessão Multibanco”:
 - - Data/Hora de Início
 - - Data/Hora de Fim
 - - Conta
 - - Transações Efetuadas [] (múltiplas por sessão)
 - Tipo
 - Valor
 - Conta Origem
 - Conta Destino
- Ex: Funcionalidades de uma “Sessão Multibanco”:
 - Abrir Sessão
 - Transacionar (Levantar, Transferir, etc.)
 - Fechar Sessão

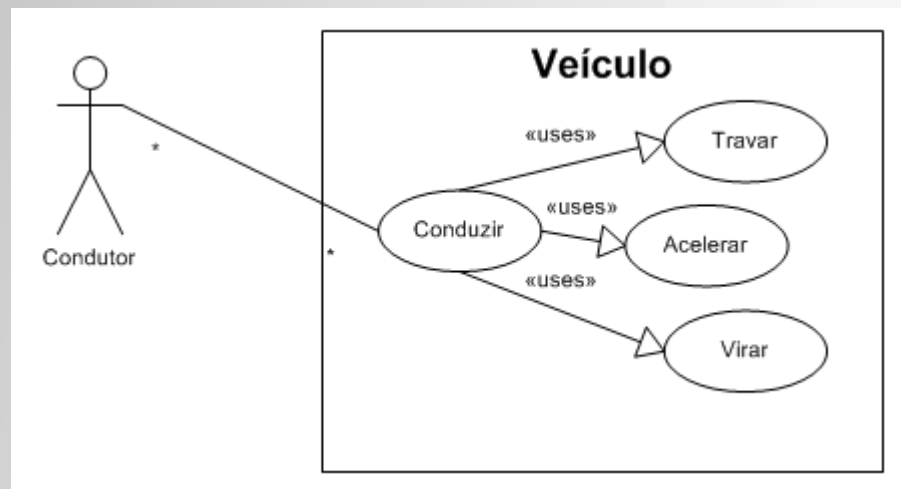
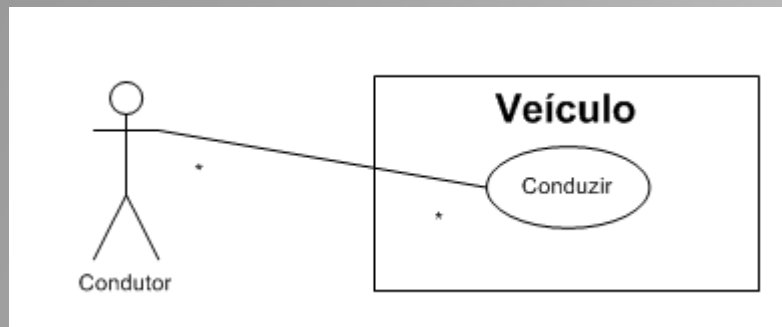
Exercício:

- Criar um Diagrama de Classes para uma clinica normal
 - *Relacionar Objetos*
 - *Enumerar alguns Atributos*
 - *Enumerar algumas Funcionalidades*
- Criar DC para uma Biblioteca Municipal
 - *Relacionar Objetos*
 - *Enumerar alguns Atributos*
 - *Enumerar algumas Funcionalidades*
- Criar DC para uma Livraria
 - *Relacionar Objetos*
 - *Enumerar alguns Atributos*
 - *Enumerar algumas Funcionalidades*

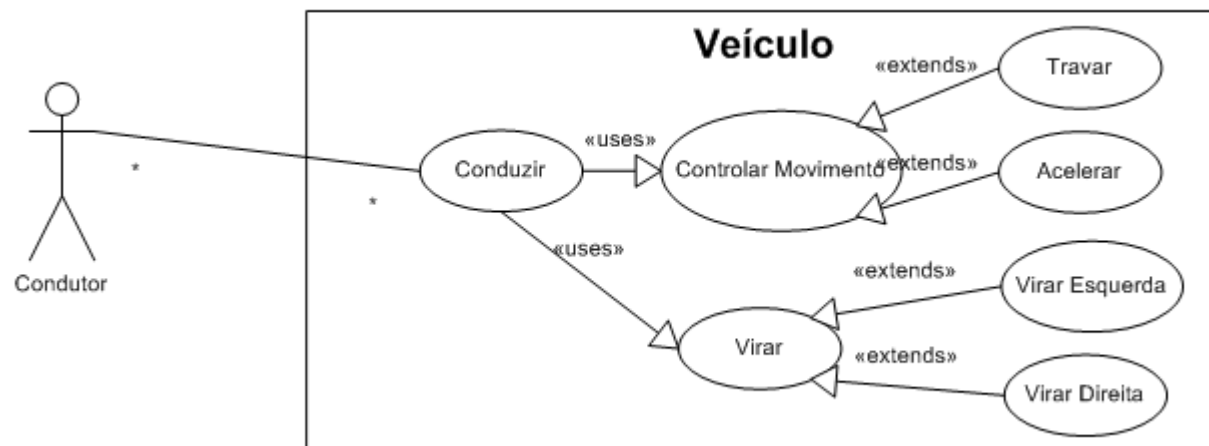
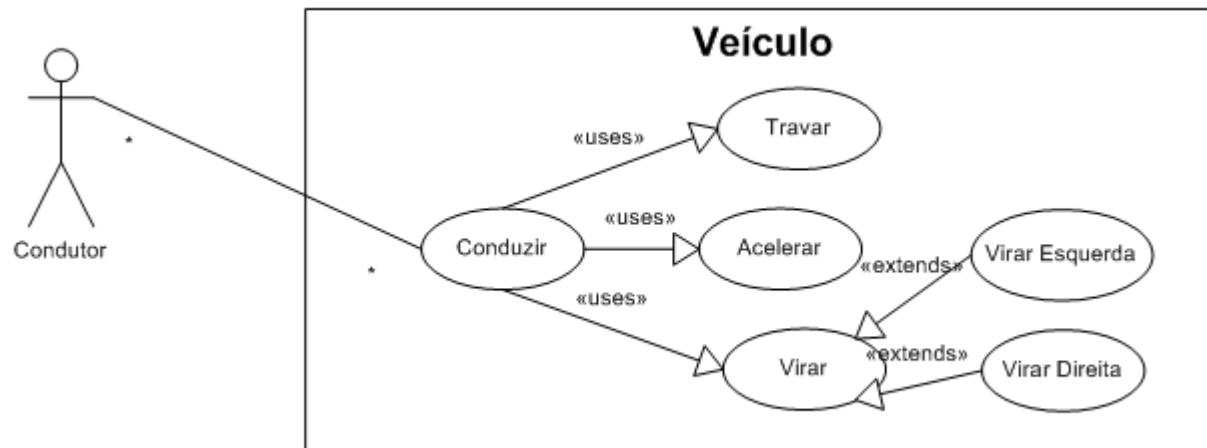
Diagramas de Casos de Uso

- Descrevem funcionalidades (interacções) do sistema
- Quem pode fazer e o quê com o sistema
 - Sempre do ponto de vista do actor
 - Vários níveis de detalhe
- Compostos por:
 - Um ou mais actores
 - Sistema(s)
 - Funcionalidades (interacções)
- Escrever um “Use Case”
 - 1. Identificar os possíveis atores de um sistema
 - 2. Escolher um dos atores
 - 3. Criar um caso de uso para cada funcionalidade que o ator pode invocar no sistema
 - 4. Enquanto houver atores, saltar para 2

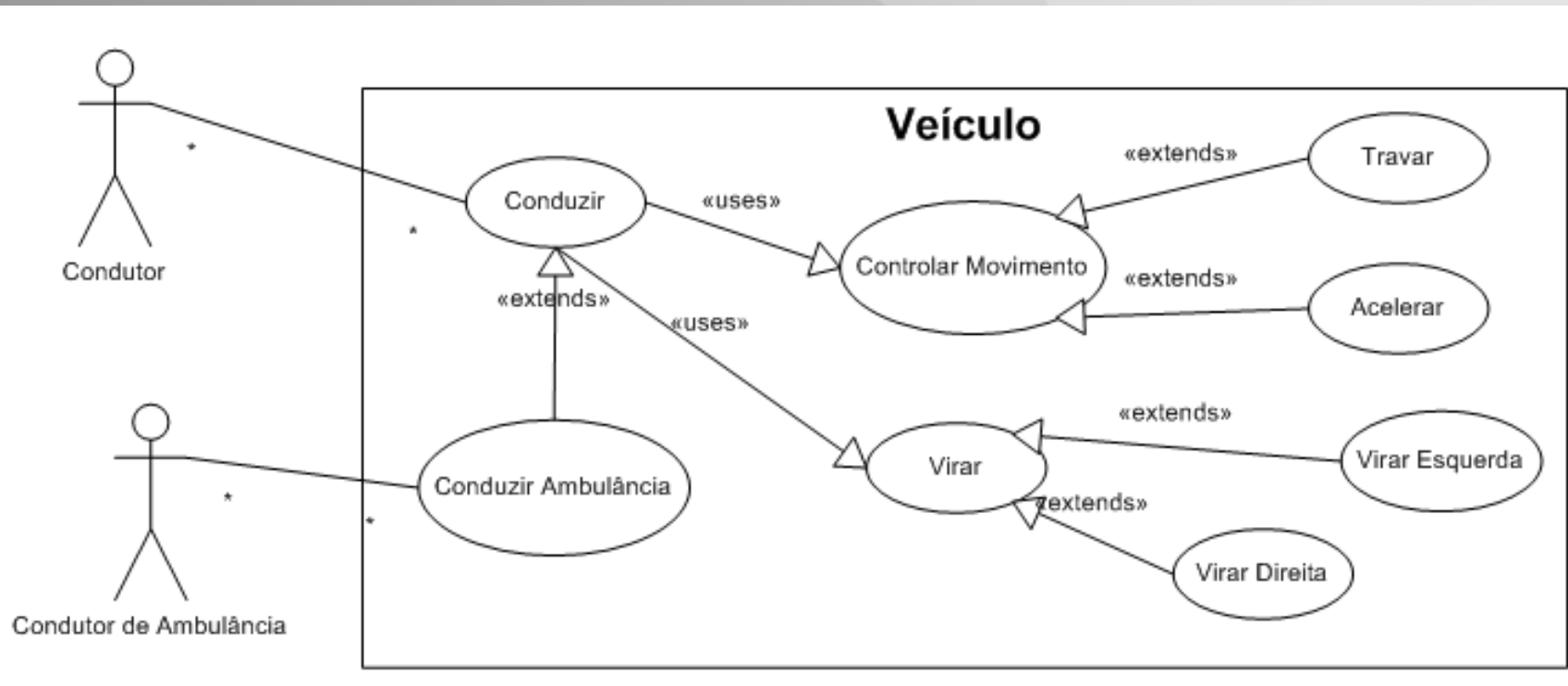
Evolução do detalhe de Caso de Uso



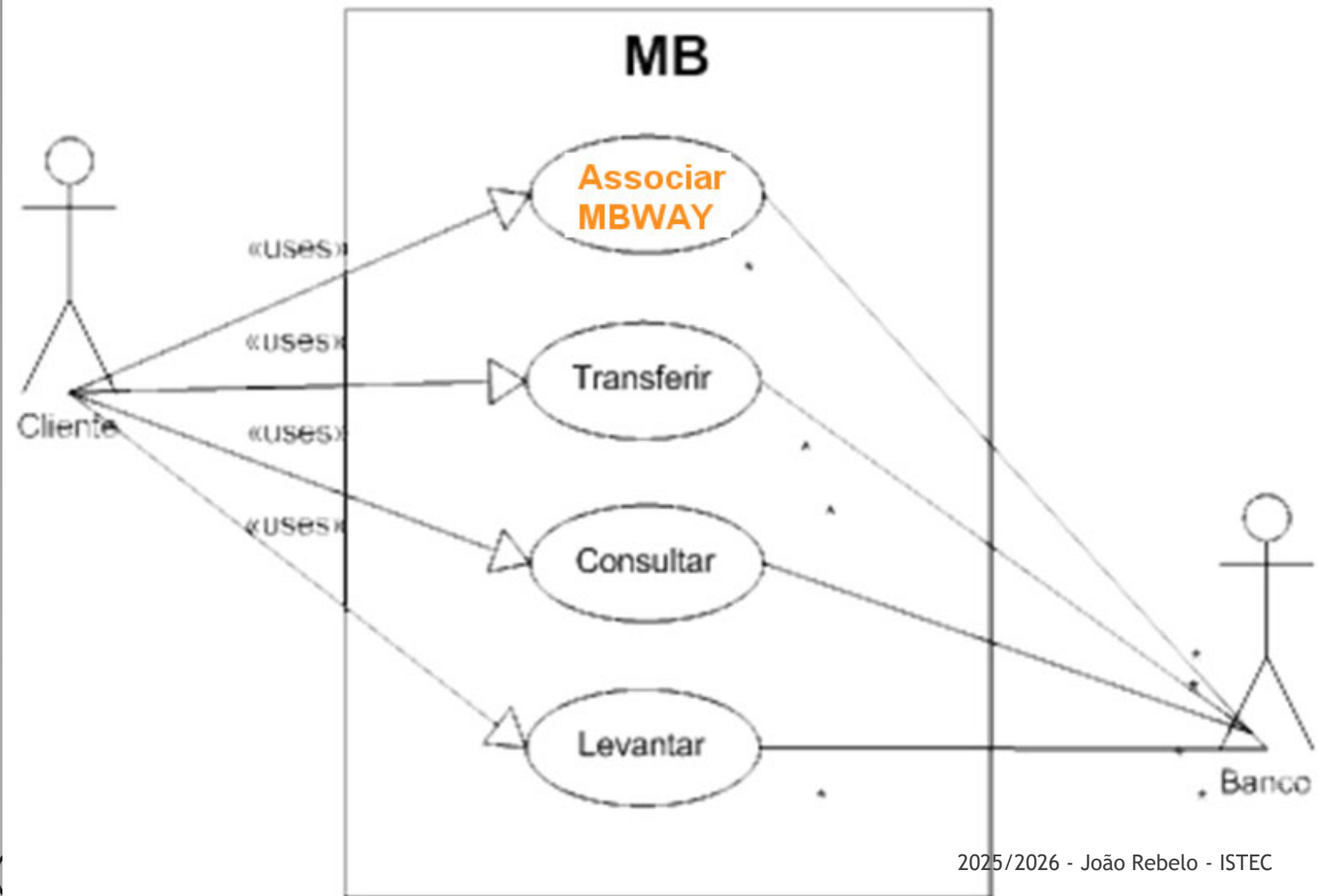
Evolução do detalhe de Caso de Uso



Evolução do detalhe de Caso de Uso



Um possível Modelo de Use Case - MB



Exercício:

- Criar UC para uma clinica normal
- Criar UC para uma Biblioteca Municipal
- Criar UC para uma Livraria

Exercício 4: Submeter no Moodle

- Elabore o descritivo DC e UC de uma solução para gerir um museu, sabendo que o visitante compra um bilhete, válido para visitar qualquer exposição em curso.
- Dado ser um museu avançado, cada obra/peça exposta tem um dispositivo IoT que deteta o bilhete do utilizador e regista essa visita à obra/peça.

Cenas dos Próximos Capítulos:

- Controlo de Atributos por Propriedades
- *Overloading*
- Programar Associações, Composições e Agregações

Dúvidas e Sugestões

- jrebelo@my.istec.pt

(João Rebelo)